Toktar Sultan
Master's Student, School of Software Engineering
Astana IT University

Assignment 5
DevOps and CI/CD: Automating Processes for Research Projects

The Role of DevOps and CI/CD in Automating Research Projects

This note is about the public-sentiment project: reading Kazakh and Russian comments on national policy and marking them positive, negative, or neutral. The thesis compared classical models with Gemini 3.5 Flash and kept Gemini. Assignment 5 does not repeat that comparison. It asks a smaller question. After the code changes, how does the same scoring service get tested, packed, and started again without someone doing those steps by hand?

1 What DevOps is

DevOps is a way of working, not a single program. The name joins development and operations. In a company those are often two teams. In a research group they are often the same two people, and the work still splits in practice. One week is spent on the model. The next week is spent trying to remember which library version produced the table in the draft.

The aim is to make a change small, check it soon, and leave a record of how it was run. A few habits follow from that.

- Keep the change small enough that a failure still points at one commit.
- Write the setup down, so the next run does not depend on a laptop that happens to have the right packages.
- Let a script do the steps a person would forget on a Friday evening.
- Look at the result of those steps before calling the change finished.

The gain for this project is practical. A word-list edit can flip a label. If the tests run on their own, that flip shows up before the demo. A new student can start the service from the same image instead of rebuilding a Python environment from a chat message.

2 CI and CD

Continuous integration means that a shared branch is built and tested whenever it changes. The point is the delay. A mistake found ten minutes later is still easy to undo. The same mistake found after the numbers have gone into a draft is not.

Continuous delivery goes one step further. The tested code is packed into something that could be released. A person still decides when it goes live. Continuous deployment would skip that person and send every green build to users. This project stops at delivery. Jenkins builds a Docker image and can place it on a local Kubernetes cluster. Nothing is published to the public internet, and the image is not pushed to Docker Hub, because no Hub account was connected for this assignment.

The split matters in research. A green test means the scorer still agrees with the small set of comments we wrote by hand. It does not mean the service should be opened to the public. Those are different decisions, and the pipeline keeps them apart.

3 What this changes for the sentiment work

The thesis code is ordinary Python: comments in, a label out. The trouble starts around that code.

The first trouble is the machine. One laptop has Python 3.12 and a new FastAPI. Another has an older pip and an older Pydantic. The service starts on one and fails on the other, and the failure has nothing to do with Kazakh. Docker is the written-down machine. The Dockerfile names Python 3.12 and the libraries. The container that Jenkins builds is the container the cluster runs.

The second trouble is a quiet change in the scorer. The labels depend on a short list of words. “дұрыс емес” must stay negative, and it must not also count as the positive word “дұрыс”. A unit test locks that case. Jenkins runs the tests before it builds the image. If the test fails, there is no new image.

The third trouble is the key. Gemini 3.5 Flash is called with an API key. That key does not belong in Git, and it does not belong in a build log. The service in this assignment is the local scorer, so the pipeline has nothing to leak. When the live call is turned on later, the key stays in an environment variable on the machine that runs it, not in the repository.

A research group also loses time on the demo itself. Packing the service once, then starting that pack, is less fragile than installing packages on the day of the presentation.

4 Docker, Jenkins, and Kubernetes

Docker packages the process. The image for this service starts from `python:3.12-slim`, installs the runtime libraries, copies the scoring code, and listens on port 8000. Tests are not installed in that image. They run earlier, in the pipeline, from a separate requirements file. The image stays a running service, not a copy of the whole repository.

People share research code this way already. Binder runs a notebook from a Docker image so that a reader gets the same libraries as the author. The sentiment service is the same idea, with a smaller job: one HTTP process instead of a notebook.

JupyterHub is the version a group uses when many people need the same setup. The Zero to JupyterHub guide puts it on Kubernetes, and each user gets a container. A teaching lab can restart that container without asking who last installed a package. This API has one process, not a hub, but the reason matches. The person who wrote the scorer should not be the only person who can start it.

Groups in bioinformatics already work this way at a larger scale. nf-core publishes Nextflow pipelines whose tools are containers. A cluster, often Kubernetes, runs those containers, so the same workflow can move from a laptop to a shared machine. This project has one container. The pattern is the one those groups use: pin the image, keep the description in Git, and let the scheduler start it.

Jenkins is the server that runs the steps. It is an old tool, which is useful, because the documentation is easy to find. For this assignment it runs in a container on the same laptop as Docker Desktop. The job is a `Jenkinsfile` in the GitHub repository. On `main` it does four things, and the last two are allowed to skip.

1. Run the unit tests inside a clean Python 3.12 container.
2. Build `qogam-sentiment:latest`.
3. Push the image to Docker Hub, only if a user and a token are set. They are not set here.
4. Deploy to the local kind cluster, only if that cluster is running.

A laptop Jenkins cannot take a GitHub webhook, because GitHub cannot reach a private address. The job polls `main` every two minutes, and it can also be started by hand. That is weaker than a webhook, and it is the setup that actually works on this machine. The useful part is the log. If a label in a draft is questioned later, the build record says which commit was tested and what the tests printed. A chat message does not keep that record.

Kubernetes keeps a container running. If the process exits, the cluster starts it again. The manifest for this service asks for one copy, checks `/api/health`, and exposes the port inside the cluster. Kind is a Kubernetes cluster whose nodes are Docker containers. It is enough to show the step. It is not a lab cluster with many users.

Kubeflow is a larger example of the same idea: machine-learning jobs scheduled on Kubernetes. This project does not need that. One replica of a small API is the whole deployment. The useful part is the habit. The file that describes the running service is in Git next to the code, so the next person does not have to guess the port or the health check.

Loading the image took an extra step. Kind does not see images that exist only in Docker Desktop. The deploy script saves the image and imports it into the kind node with `ctr`, then runs `kubectl apply`. After that, `imagePullPolicy: IfNotPresent` can use the local image. Without the import, Kubernetes would try Docker Hub, fail, and the pod would stay pending.

5 Conclusion

DevOps does not choose the sentiment model. Gemini 3.5 Flash was chosen in the thesis, and this pipeline does not reopen that choice. What it does is shorten the path from a code change to a running copy, and it stops that path when the tests fail.

For a scientific team the problems are familiar. Environments drift. A small edit changes a result and nobody notices until later. The steps to start the demo live in one person's head. Secrets leak into chat logs. Docker, Jenkins, and a small Kubernetes cluster answer those problems in a plain way: one image, one job that runs the tests, and one manifest that starts the image. The cluster here has a single node and a single replica. The point of the exercise is that the same files would be the start of a larger setup, not that this laptop is a production platform.